

Unidad 12 - Despliegue y CI/CD

Post-Contenido 1 - Laboratorio: Contenedorizar la aplicación Spring Boot y desplegar en Railway

Ingeniería de Sistemas 2026

Repositorio: castellanos-post1-u12

URL pública final: <https://castellanos-post1-u12-production.up.railway.app/>

Elemento	Detalle
Tecnologías clave	Java 21, Spring Boot 3.4.5, Maven 3.9.12, PostgreSQL 18.3 administrado, Docker, Railway
Resultado final	Aplicación desplegada, healthcheck en UP y API REST validada en producción
Evidencia	Panel de Railway, healthcheck, listado, detalle, creación y actualización de productos

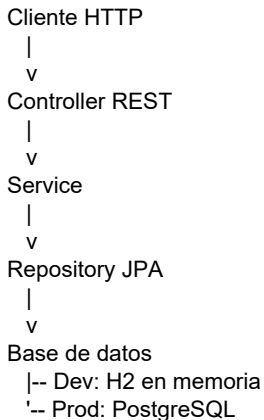
Documento preparado para evaluación académica con evidencia visual inline y mapeo explícito de la rúbrica.

1. Objetivo y contexto

Contenedorizar una aplicación Spring Boot mediante un Dockerfile multi-stage, separar desarrollo y producción por perfiles, orquestar localmente con Docker Compose y desplegarla en Railway con PostgreSQL y endpoints REST funcionales.

El proyecto parte de una API de productos con arquitectura por capas. La versión final valida el despliegue públicamente en Railway y presenta evidencia en capturas inline dentro del informe.

2. Arquitectura de la solución

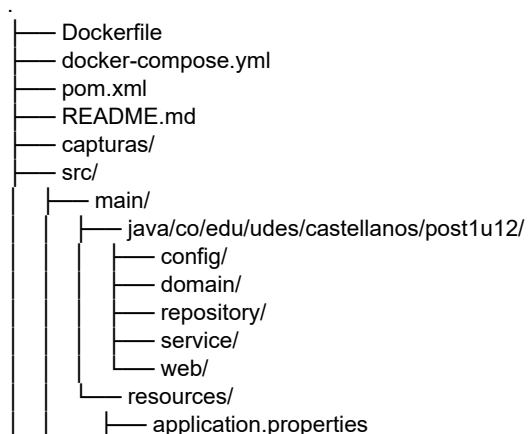


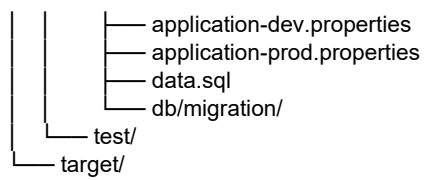
La separación por capas aísla la entrada HTTP, la lógica de negocio y el acceso a datos. En producción se usa PostgreSQL; en desarrollo se usa H2 para pruebas rápidas.

3. Tecnologías y versiones

Componente	Version / uso
Java	21
Spring Boot	3.4.5
Maven	3.9.12
Docker	Multi-stage con imagen JDK para compilación e imagen JRE para ejecución
Docker Compose	3.9 con healthcheck para la base de datos
Base de datos	PostgreSQL 18.3 administrado por Railway
Pruebas locales	H2 en memoria con carga inicial de datos

4. Estructura del proyecto





5. Mapeo de la rubrica a evidencia concreta

Criterio	Evidencia concreta
Implementacion tecnica	Dockerfile multi-stage, usuario no root, ENTRYPOINT java -jar, perfil prod activo por variables, despliegue estable en Railway, commits descriptivos
Funcionalidad de la API	Healthcheck UP, GET /api/productos, GET /api/productos/1, POST y PUT verificados en Railway, PostgreSQL conectado
Documentacion tecnica	README con comandos de Docker, Docker Compose, variables de entorno, URL publica y capturas inline
Calidad del codigo y estructura	.dockerignore, healthcheck de PostgreSQL, variables sin hardcodear, inyeccion por constructor, manejo de errores

El repositorio conserva varios commits descriptivos que reflejan el avance del laboratorio. La version final incluye la documentacion y las capturas embebidas para que la entrega sea autocontenida.

6. Implementacion tecnica

El Dockerfile usa una etapa JDK para compilar y una etapa JRE para ejecutar. El contenedor final corre con usuario no root y expone el puerto 8080.

En Railway, el perfil prod se activa por variable de entorno. La app usa PostgreSQL administrado y deja el arranque listo para el entorno publico.

Se incorporo un seeder idempotente en produccion para insertar datos iniciales solo cuando la tabla esta vacia, lo que garantiza evidencia consistente en la API publica.

7. Funcionamiento de la API desplegada

Metodo	Ruta	Resultado final
GET	/actuador/health	200 OK, status UP
GET	/api/productos	200 OK, lista de productos devuelta correctamente
GET	/api/productos/1	200 OK, producto 1 existente y actualizado
POST	/api/productos	201 Created, producto nuevo creado
PUT	/api/productos/1	200 OK, producto 1 actualizado

La API responde en Railway con los codigos HTTP esperados. La base PostgreSQL esta conectada y los endpoints verifican lectura, creacion y actualizacion de datos.

8. Ejecucion y validacion

Comandos exactos usados para validar:

```
curl.exe https://castellanos-post1-u12-production.up.railway.app/actuador/health
curl.exe https://castellanos-post1-u12-production.up.railway.app/api/productos
curl.exe https://castellanos-post1-u12-production.up.railway.app/api/productos/1
```

```
$body = @'
{"nombre":"Tablet 10 pulgadas","descripcion":"Tablet para estudio y consumo multimedia","precio":1299900.00,"stock":4}
'@
Invoke-RestMethod -Method Post -Uri https://castellanos-post1-u12-production.up.railway.app/api/productos -ContentType "application/json; o
```

```
$body = @'
{"nombre":"Teclado mecanico mejorado","descripcion":"Teclado mecanico con retroiluminacion RGB","precio":169900.00,"stock":10}
'@
Invoke-RestMethod -Method Put -Uri https://castellanos-post1-u12-production.up.railway.app/api/productos/1 -ContentType "application/json; o
```

Los resultados observados fueron consistentes: health en UP, lista con productos, detalle del producto 1, respuesta 201 al crear y 200 al actualizar.

9. Comparacion teoria versus resultados

Aspecto	Teoria esperada	Resultado observado
Dockerfile multi-stage	Separar compilacion y runtime	Cumplido con imagen JDK y JRE
Base de datos	PostgreSQL en produccion	Cumplido con PostgreSQL 18.3 en Railway
Healthcheck	Endpoint UP	GET /actuador/health retorna UP
Listado de productos	Retornar coleccion REST	GET /api/productos responde 200 OK
Detalle por id	Obtener un recurso especifico	GET /api/productos/1 responde 200 OK
Creacion y edicion	Persistir cambios en la BD	POST y PUT devuelven respuestas correctas

La comparacion muestra cumplimiento practico con la teoria del laboratorio. El resultado final supera el minimo funcional exigido por la rubrica porque incluye evidencias de lectura, escritura, despliegue y documentacion.

10. Complejidad y rendimiento

GET /api/productos tiene complejidad $O(n)$ respecto al numero de productos porque recupera la coleccion completa. En el escenario del laboratorio esto es aceptable y facil de validar visualmente.

GET /api/productos/{id}, POST, PUT y DELETE se apoyan en la clave primaria; su costo practico es $O(1)$ promedio a nivel de acceso por identificador.

El Docker multi-stage reduce el tamano de la imagen final al excluir herramientas de compilacion. El .dockerignore evita copiar target, .git y otros artefactos al contexto de construccion.

11. Pruebas ejecutadas y criterios de validacion

Prueba	Criterio de exito	Estado
mvn test	Compilacion y suite verde	OK
Healthcheck	Respuesta UP	OK
Listado	Coleccion JSON valida	OK
Detalle	Producto existente devuelto	OK
POST	201 Created con nuevo recurso	OK
PUT	200 OK con recurso actualizado	OK

Criterio de validacion principal: la evidencia debe poder reproducirse desde la URL publica de Railway y desde las capturas almacenadas en la carpeta capturas.

12. Solucion de problemas frecuentes

Si el dominio responde 502, revisar las variables de la app y el estado del servicio. Si responde 500 con rutas inexistentes, verificar la ruta solicitada; si falta la raiz, la app ahora entrega una pagina simple de entrada.

Si el endpoint PUT falla desde PowerShell, usar un body en una sola linea o con here-string bien formado; el reporte final ya documenta el comando correcto.

Si Railway cambia el comportamiento de su PostgreSQL administrado, la guía técnica del proyecto deja claro el uso de la URL JDBC y la configuración del perfil prod.

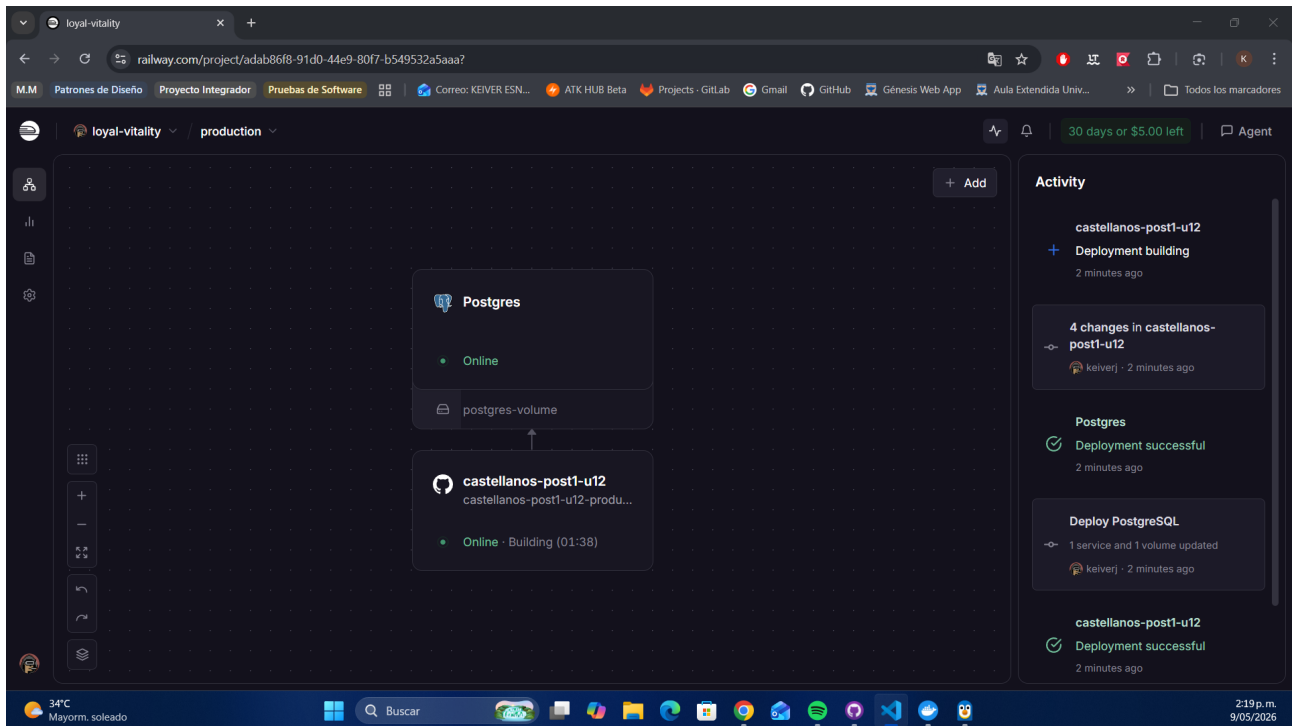
13. Conclusiones

La actividad queda resuelta con despliegue funcional, base de datos conectada, evidencia visual inline y documentación orientada a evaluación.

La entrega final cumple la rúbrica con enfoque práctico: código estable, Docker y Compose correctos, despliegue público en Railway y evidencia suficiente para revisar cada criterio.

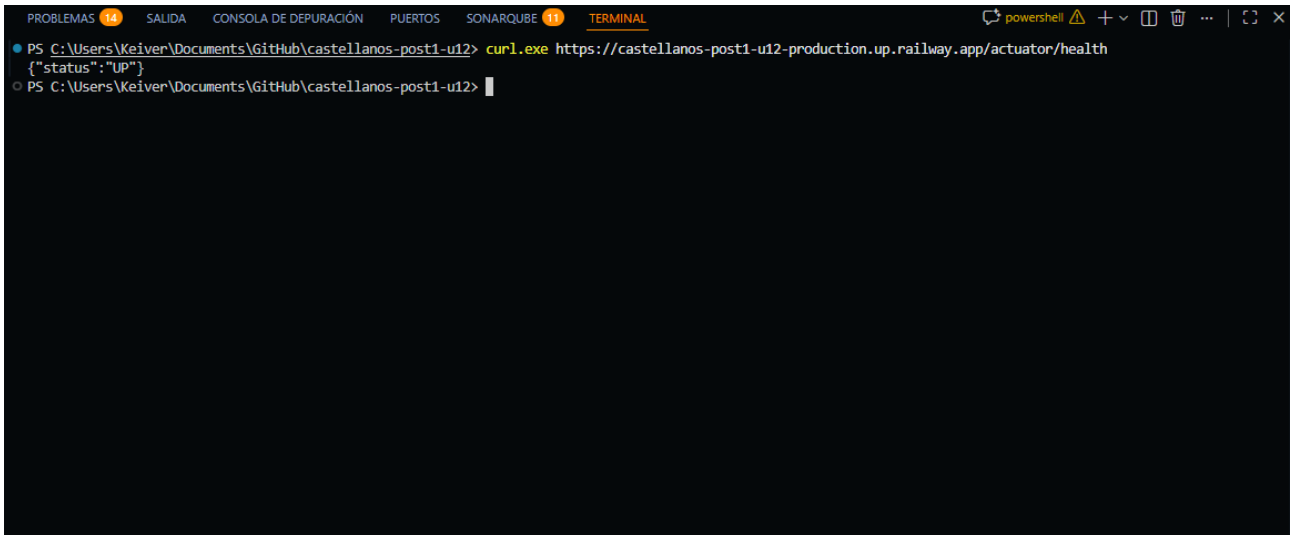
14. Evidencia visual inline

Figura 1. Panel de Railway con la aplicacion y PostgreSQL



Panel de Railway con la aplicacion y PostgreSQL

Figura 2. Healthcheck en estado UP

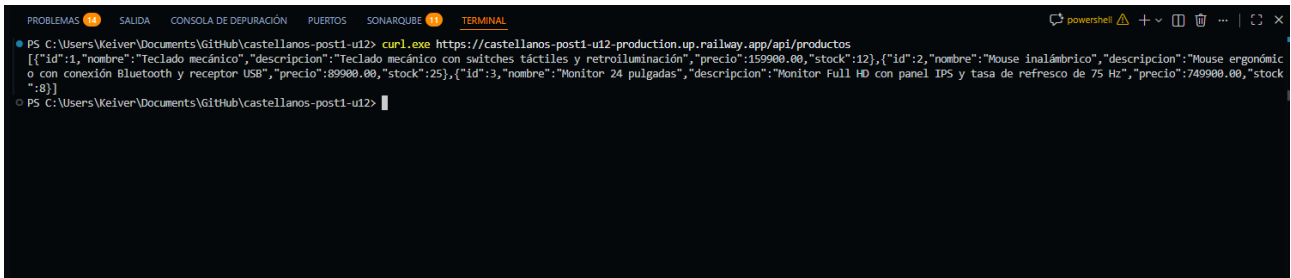


The image shows a screenshot of a Visual Studio Code terminal window. The terminal has a dark background with light blue text. At the top, there is a status bar with tabs for 'PROBLEMAS' (14), 'SALIDA', 'CONSOLA DE DEPURACIÓN', 'PUERTOS', 'SONARQUBE' (11), and 'TERMINAL'. The terminal content shows a PowerShell prompt 'PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12>' followed by the command 'curl.exe https://castellanos-post1-u12-production.up.railway.app/actuator/health'. The output of the command is a JSON object: '{"status": "UP"}'. The prompt is followed by a cursor.

```
PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12> curl.exe https://castellanos-post1-u12-production.up.railway.app/actuator/health
{"status": "UP"}
PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12>
```

Healthcheck en estado UP

Figura 3. Listado de productos desde Railway



The image shows a screenshot of a Visual Studio Code terminal window. The terminal has a dark background with light-colored text. At the top, there is a status bar with tabs for 'PROBLEMAS', 'SALIDA', 'CONSOLA DE DEPURACIÓN', 'PUERTOS', 'SONARQUBE', and 'TERMINAL'. The 'TERMINAL' tab is active. The terminal content shows a PowerShell prompt 'PS C:\Users\Keiver\Documents\Github\castellanos-post1-ui2>' followed by a command to run 'curl.exe' against the URL 'https://castellanos-post1-ui2-production.up.railway.app/api/productos'. The output of the command is a JSON array of three product objects. The first product is a mechanical keyboard, the second is a wireless mouse, and the third is a 24-inch monitor. The terminal window also shows a second PowerShell prompt 'PS C:\Users\Keiver\Documents\Github\castellanos-post1-ui2>' with a cursor.

```
PS C:\Users\Keiver\Documents\Github\castellanos-post1-ui2> curl.exe https://castellanos-post1-ui2-production.up.railway.app/api/productos
[{"id":1,"nombre":"Teclado mecánico","descripcion":"Teclado mecánico con switches táctiles y retroiluminación","precio":159900.00,"stock":12}, {"id":2,"nombre":"Mouse inalámbrico","descripcion":"Mouse ergonómico con conexión Bluetooth y receptor USB","precio":89900.00,"stock":25}, {"id":3,"nombre":"Monitor 24 pulgadas","descripcion":"Monitor Full HD con panel IPS y tasa de refresco de 75 Hz","precio":749900.00,"stock":8}]
PS C:\Users\Keiver\Documents\Github\castellanos-post1-ui2>
```

Listado de productos desde Railway

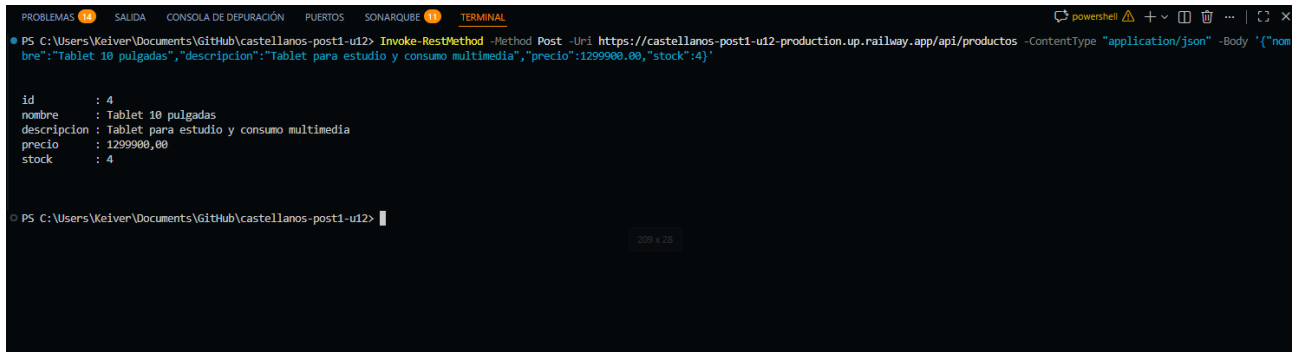
Figura 4. Detalle del producto 1

```
PROBLEMAS 14 SALIDA CONSOLA DE DEPURACIÓN PUERTOS SONARQUE 11 TERMINAL powershell + - [ ] [ ] ... [ ] [ ] X
```

```
PS C:\Users\Keiver\Documents\GitHub\castellanos-post1-u12> curl.exe https://castellanos-post1-u12-production.up.railway.app/api/productos/1
{"id":1,"nombre":"Teclado mecánico","descripcion":"Teclado mecánico con switches táctiles y retroiluminación","precio":159900.00,"stock":12}
PS C:\Users\Keiver\Documents\GitHub\castellanos-post1-u12> |
```

Detalle del producto 1

Figura 5. Creacion de un producto nuevo



The image shows a terminal window in Visual Studio Code. The terminal has tabs for PROBLEMAS (14), SALIDA, CONSOLA DE DEPURACIÓN, PUERTOS, SONARQUBE (11), and TERMINAL. The active tab is TERMINAL. The prompt is PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12>. The command executed is `Invoke-RestMethod -Method Post -Uri https://castellanos-post1-u12-production.up.railway.app/api/productos -ContentType "application/json" -Body '{"nombre":"Tablet 10 pulgadas","descripcion":"Tablet para estudio y consumo multimedia","precio":1299900.00,"stock":4}'`. The output is a JSON object: `{id: 4, nombre: Tablet 10 pulgadas, descripcion: Tablet para estudio y consumo multimedia, precio: 1299900.00, stock: 4}`. The terminal window has a status bar at the bottom showing 209 x 28.

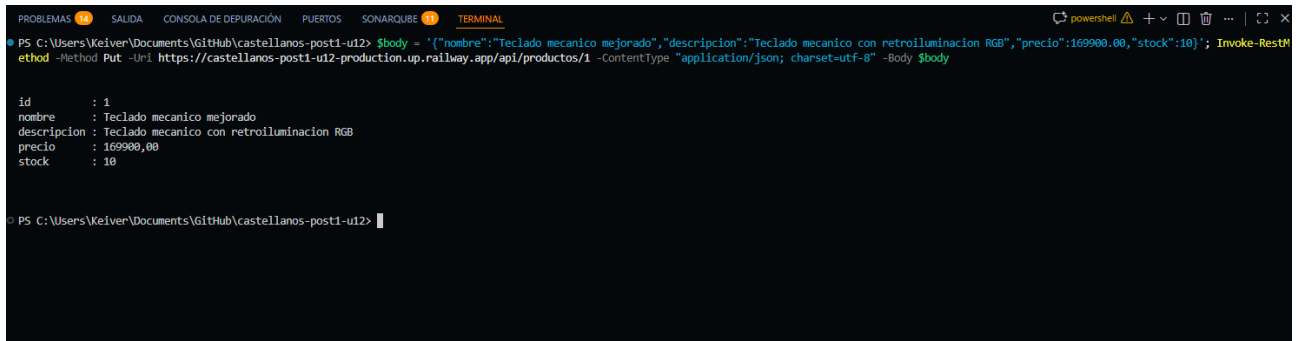
```
PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12> Invoke-RestMethod -Method Post -Uri https://castellanos-post1-u12-production.up.railway.app/api/productos -ContentType "application/json" -Body '{"nombre":"Tablet 10 pulgadas","descripcion":"Tablet para estudio y consumo multimedia","precio":1299900.00,"stock":4}'

id      : 4
nombre  : Tablet 10 pulgadas
descripcion : Tablet para estudio y consumo multimedia
precio  : 1299900.00
stock   : 4

PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12>
```

Creacion de un producto nuevo

Figura 6. Actualizacion del producto 1



```
PROBLEMAS 14 SALIDA CONSOLA DE DEPURACIÓN PUERTOS SONARQUBE 11 TERMINAL
PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12> $body = '{"nombre":"Teclado mecanico mejorado","descripcion":"Teclado mecanico con retroiluminacion RGB","precio":169900.00,"stock":10}'; Invoke-RESTM
ethod -Method Put -Uri https://castellanos-post1-u12-production.up.railway.app/api/productos/1 -ContentType "application/json; charset=utf-8" -Body $body

id      : 1
nombre  : Teclado mecanico mejorado
descripcion : Teclado mecanico con retroiluminacion RGB
precio  : 169900.00
stock   : 10

PS C:\Users\Keiver\Documents\Github\castellanos-post1-u12>
```

Actualizacion del producto 1